🩺 Haberman's Survival Data

Exploratory Data Analysis

Can a patient's age, the year of her operation or the number of positive lymph nodes tell us whether she survives 5 years after breast-cancer surgery?

What this notebook does: it explores 306 patients from a 1958–1970 study at the University of Chicago's Billings Hospital, first one feature at a time (univariate analysis) and then in pairs and triples (bivariate analysis), to find out which features, if any, separate patients who **survived 5 years or longer** from those who **died within 5 years**.

Part	Content
1	Dataset details
2	Setup, loading and data-quality checks
3	Class balance and per-class summary
4	Univariate analysis: PDF, CDF, box plot, violin plot, 1D scatter
5	Bivariate analysis: 2D scatter, 3D scatter, pair plot, correlation, density contours
6	Key findings, observations and conclusions

1 · Dataset details

Title	Haberman's Survival Data
Donor	Tjen-Sien Lim (limt@stat.wisc.edu), March 4, 1999
Study	1958–1970, University of Chicago's Billings Hospital, patients who had surgery for breast cancer
Instances	306
Attributes	4 (3 features + 1 class label)
Missing values	None

Column	Meaning	Type
`age`	Age of the patient at the time of the operation	numerical
`operation_year`	Year of the operation (year − 1900)	numerical
`axil_nodes`	Number of positive axillary lymph nodes detected	numerical
`survival_status`	`1` = survived 5 years or longer, `2` = died within 5 years (class label)	categorical

▼ Past usage of this dataset

- Haberman, S. J. (1976). *Generalized Residuals for Log-Linear Models*, Proceedings of the 9th International Biometrics Conference, Boston, pp. 104–122.
- Landwehr, J. M., Pregibon, D., and Shoemaker, A. C. (1984). *Graphical Models for Assessing Logistic Regression Models*, JASA 79: 61–83.
- Lo, W.-D. (1993). *Logistic Regression Trees*, PhD thesis, University of Wisconsin, Madison.

2 · Setup, loading and data-quality checks

Requirements: `pip install pandas numpy matplotlib seaborn plotly nbformat kaleido` (seaborn ≥ 0.13 is recommended). `kaleido` is only needed to save the 3D plot as a static image.

The next cell imports the libraries and defines one **colour scheme and chart style** used by every plot below: teal = survived 5+ years (`yes`), coral = died within 5 years (`no`).

```
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.ticker import PercentFormatter
import seaborn as sns
import plotly.express as px
from IPython.display import Image, display

warnings.filterwarnings("ignore", category=FutureWarning)
warnings.filterwarnings("ignore", category=UserWarning)

# ── Colours: one meaning per colour, used in every chart ──────────────
YES, NO, INK, MUTED = "#2A9D8F", "#E76F51", "#264653", "#6B7280"
PALETTE = {"yes": YES, "no": NO}
ORDER = ["yes", "no"]
NICE = {"yes": "Survived 5+ years", "no": "Died within 5 years"}

FEATURES = ["age", "operation_year", "axil_nodes"]
TARGET = "survival_status"
FEATURE_LABEL = {
    "age": "Age at operation (years)",
    "operation_year": "Year of operation (19xx)",
    "axil_nodes": "Positive axillary nodes",
}

# ── Chart style ──────────────────────────────────────────────────────
sns.set_theme(style="whitegrid")
plt.rcParams.update({
    "figure.dpi": 110, "savefig.dpi": 200, "figure.facecolor": "white",
    "font.size": 10.5, "axes.titlesize": 12.5, "axes.titleweight": "bold",
    "axes.titlelocation": "left", "axes.titlecolor": INK, "axes.labelsize": 10.5,
    "axes.labelcolor": "#374151", "axes.edgecolor": "#D1D5DB",
    "axes.spines.top": False, "axes.spines.right": False,
    "grid.color": "#E5E7EB", "grid.linewidth": 0.8, "axes.axisbelow": True,
    "legend.frameon": False, "legend.fontsize": 9.5, "xtick.color": "#4B5563", "ytick.color": "#4B5563",
})

def nice_legend(legend, title=None):
```

```
    """Replace the raw 'yes'/'no' legend entries with readable text."""
    if legend is None:
        return
    legend.set_title(title)
    for text in legend.get_texts():
        text.set_text(NICE.get(text.get_text(), text.get_text()))

def headline(fig, text):
    """Left-aligned figure headline."""
    fig.suptitle(text, x=0.01, ha="left", fontsize=14.5, fontweight="bold", color=INK, y=1.02)
```

`Haberman.csv` has **no header row**, so `header=None` is required. Without it, pandas would treat the first patient as the header and silently drop them (305 rows instead of 306).

```
haberman = pd.read_csv("Haberman.csv", header=None, names=FEATURES + [TARGET])
print(f"Rows: {haberman.shape[0]}   Columns: {haberman.shape[1]}")
haberman.head()
```

```
Rows: 306   Columns: 4
```

	age	operation_year	axil_nodes	survival_status
0	30	64	1	1
1	30	62	3	1
2	30	65	0	1
3	31	59	2	1
4	31	65	4	1

```
haberman.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 306 entries, 0 to 305
Data columns (total 4 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   age              306 non-null    int64
 1   operation_year   306 non-null    int64
 2   axil_nodes       306 non-null    int64
 3   survival_status  306 non-null    int64
dtypes: int64(4)
memory usage: 9.7 KB
```

```
quality = pd.DataFrame({
    "missing values": haberman.isna().sum(),
    "unique values": haberman.nunique(),
    "min": haberman.min(),
    "max": haberman.max(),
})
display(quality)
print("Duplicate rows:", haberman.duplicated().sum())
```

	missing values	unique values	min	max
age	0	49	30	83
operation_year	0	12	58	69
axil_nodes	0	31	0	52
survival_status	0	2	1	2

```
Duplicate rows: 17
```

The data is complete: **no missing values**. There are 17 duplicate rows, which are kept because different patients can legitimately share the same age, year and node count.

```
haberman.describe().round(2)
```

	age	operation_year	axil_nodes	survival_status
count	306.00	306.00	306.00	306.00
mean	52.46	62.85	4.03	1.26
std	10.80	3.25	7.19	0.44
min	30.00	58.00	0.00	1.00
25%	44.00	60.00	0.00	1.00
50%	52.00	63.00	1.00	1.00
75%	60.75	65.75	4.00	2.00
max	83.00	69.00	52.00	2.00

The class label is stored as the integers 1 and 2. Converting it to readable categories (`yes` = survived 5+ years, `no` = died within 5 years) makes every plot and legend self-explanatory.

```
haberman[TARGET] = haberman[TARGET].map({1: "yes", 2: "no"})
haberman[TARGET] = pd.Categorical(haberman[TARGET], categories=ORDER)

print("Features:", FEATURES)
print("Class label:", TARGET)
print("Classes:", list(haberman[TARGET].cat.categories))
haberman.sample(8, random_state=42)
```

```
Features: ['age', 'operation_year', 'axil_nodes']
Class label: survival_status
Classes: ['yes', 'no']
```

	age	operation_year	axil_nodes	survival_status
182	55	58	1	yes
154	52	65	0	yes
111	48	62	2	yes
203	57	62	0	yes
60	42	65	0	yes
9	34	58	30	yes
119	49	60	1	yes
157	53	65	1	no

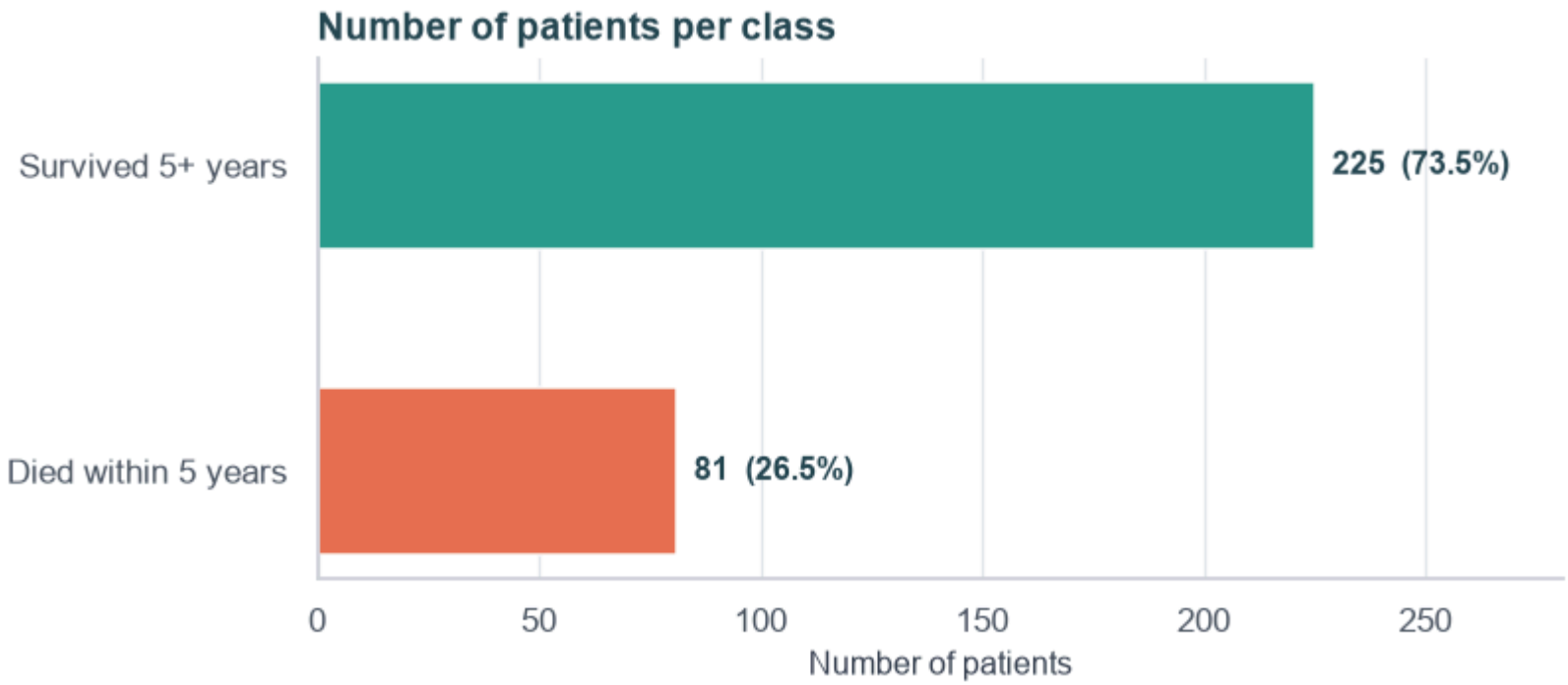
3 · Class balance and per-class summary

```
counts = haberman[TARGET].value_counts().reindex(ORDER)
summary = pd.DataFrame({"patients": counts, "share (%)": (counts / counts.sum() * 100).round(1)})
summary.index = [NICE[i] for i in summary.index]
display(summary)

fig, ax = plt.subplots(figsize=(7.5, 3.4))
bars = ax.barh(
    [NICE[i] for i in ORDER][::-1],
    counts.to_numpy(dtype=float)[::-1],
    color=[NO, YES],
    height=0.55,
)
for bar, value in zip(bars, counts.values[::-1]):
```

```
    ax.text(value + 4, bar.get_y() + bar.get_height() / 2, f"{value}  ({value / counts.sum():.1%})",
            va="center", color=INK, fontweight="bold")
ax.set(xlim=(0, counts.max() * 1.25), xlabel="Number of patients")
ax.grid(axis="y", visible=False)
ax.set_title("Number of patients per class")
plt.tight_layout()
plt.show()
```

	patients	share (%)
Survived 5+ years	225	73.5
Died within 5 years	81	26.5



Observation: the dataset is **imbalanced**: 225 patients survived (73.5%) and 81 died (26.5%). A model that always predicts "survived" would already be 73.5% accurate, so plain accuracy would be misleading here.

```
def iqr(x):
    return x.quantile(0.75) - x.quantile(0.25)
```

```
per_class = haberman.groupby(TARGET, observed=True)[FEATURES].agg(["mean", "median", "std", iqr, "min", "max"]).round(2)
per_class.index = [NICE[i] for i in per_class.index]
per_class.T
```

		Survived 5+ years	Died within 5 years
age	**mean**	52.02	53.68
	median	52.00	53.00
	std	11.01	10.17
	iqr	17.00	15.00
	min	30.00	34.00
	max	77.00	83.00
operation_year	**mean**	62.86	62.83
	median	63.00	63.00
	std	3.22	3.34
	iqr	6.00	6.00
	min	58.00	58.00
	max	69.00	69.00
axil_nodes	**mean**	2.79	7.46
	median	0.00	4.00
	std	5.87	9.19
	iqr	3.00	10.00
	min	0.00	0.00
	max	46.00	52.00

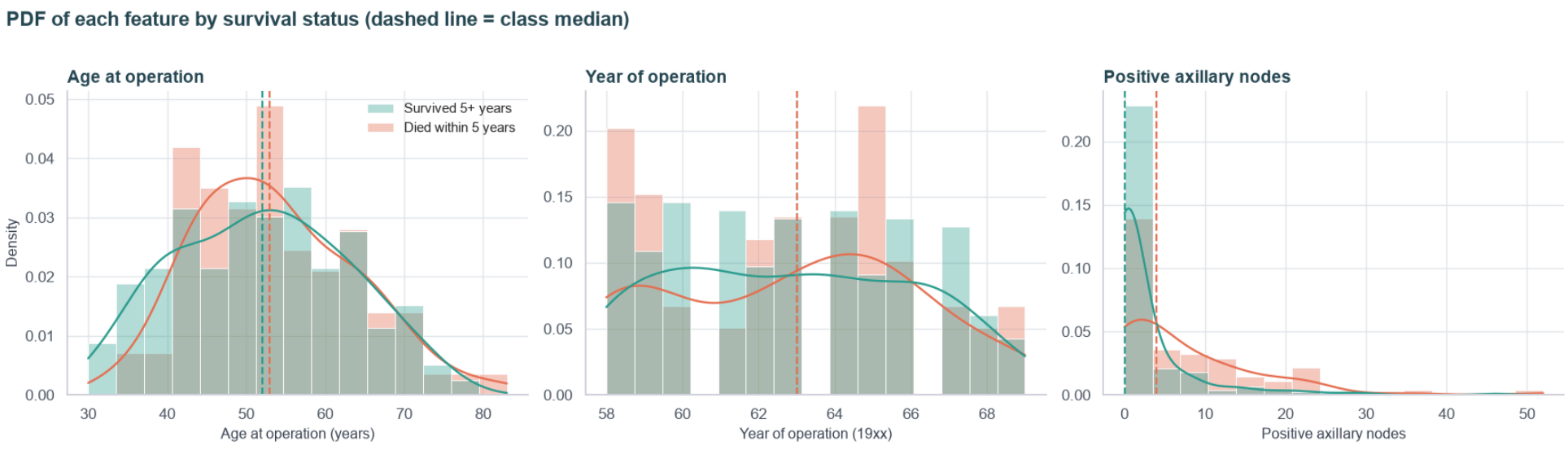
4 · Univariate analysis

Univariate analysis looks at **one variable at a time**. It summarises each feature and shows whether its distribution differs between the two classes. A feature whose distribution is clearly different for the two classes is useful for classification.

4.1 PDF: Probability Density Function

A PDF shows how the values of a *continuous* variable are spread: the area under the curve between two values is the probability of falling in that range. Below, a histogram with a smoothed density curve is drawn **separately for each class** (`common_norm=False` scales each class to itself, since the classes have different sizes). Dashed lines mark each class's median.

```
fig, axes = plt.subplots(1, 3, figsize=(16, 4.4))
for k, (ax, feature) in enumerate(zip(axes, FEATURES)):
    sns.histplot(data=haberman, x=feature, hue=TARGET, hue_order=ORDER, palette=PALETTE, stat="density",
                 common_norm=False, kde=True, bins=15, alpha=0.35, edgecolor="white", linewidth=0.6,
                 legend=(k == 0), ax=ax)
    for status in ORDER:
        median = haberman.loc[haberman[TARGET] == status, feature].median()
        ax.axvline(median, color=PALETTE[status], linestyle="--", linewidth=1.4)
    ax.set(title=FEATURE_LABEL[feature].split(" (")[0], xlabel=FEATURE_LABEL[feature], ylabel="Density" if k == 0 else "")
nice_legend(axes[0].get_legend())
headline(fig, "PDF of each feature by survival status (dashed line = class median)")
plt.tight_layout()
plt.show()
```



Observations

1. `age` and `operation_year` have nearly the same distribution for both classes, so they separate the classes poorly.

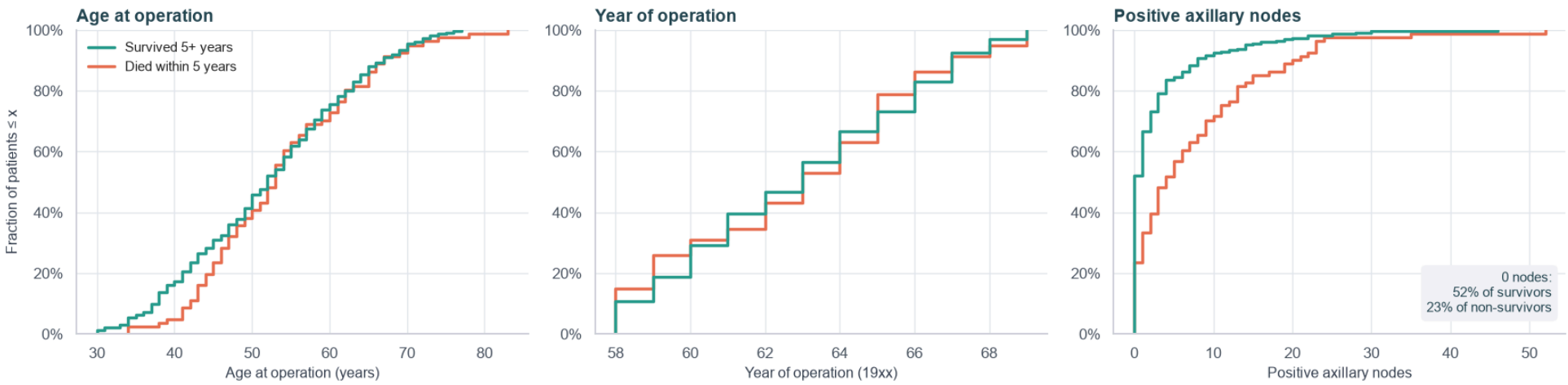
2. `axil_nodes` is clearly different: survivors are concentrated at 0–3 nodes, while patients who died have a much longer tail towards higher node counts.
3. A **median of 0 nodes** for survivors against **4 nodes** for non-survivors is the clearest difference in the data.

4.2 CDF: Cumulative Distribution Function

A CDF gives the fraction of patients with a value **less than or equal to** x. The vertical gap between the two curves shows how different the classes are at that value.

```
fig, axes = plt.subplots(1, 3, figsize=(16, 4.4))
for k, (ax, feature) in enumerate(zip(axes, FEATURES)):
    sns.ecdfplot(data=haberman, x=feature, hue=TARGET, hue_order=ORDER, palette=PALETTE,
                 linewidth=2.2, legend=(k == 0), ax=ax)
    ax.set(title=FEATURE_LABEL[feature].split(" (")[0], xlabel=FEATURE_LABEL[feature],
           ylabel="Fraction of patients ≤ x" if k == 0 else "")
    ax.yaxis.set_major_formatter(PercentFormatter(1.0))

# call-out on the node-count panel
zero = haberman.groupby(TARGET, observed=True)["axil_nodes"].apply(lambda s: (s == 0).mean())
axes[2].text(0.97, 0.06, f"0 nodes:\n{zero['yes']:.0%} of survivors\n{zero['no']:.0%} of non-survivors",
             transform=axes[2].transAxes, ha="right", va="bottom", color=INK, fontsize=10,
             bbox=dict(boxstyle="round,pad=0.4", facecolor="#F3F4F6", edgecolor="none"))
nice_legend(axes[0].get_legend())
headline(fig, "CDF of each feature by survival status")
plt.tight_layout()
plt.show()
```




```
nodes = haberman.groupby(TARGET, observed=True)["axil_nodes"]
table = pd.DataFrame({"share with 0 nodes": nodes.apply(lambda s: (s == 0).mean()),
                      "share with ≤ 5 nodes": nodes.apply(lambda s: (s <= 5).mean())})
table.index = [NICE[i] for i in table.index]
(table * 100).round(1).astype(str) + " %"
```

	share with 0 nodes	share with ≤ 5 nodes
Survived 5+ years	52.0 %	84.4 %
Died within 5 years	23.5 %	56.8 %

Observations

1. The CDFs of `age` and `operation_year` almost overlap, which confirms they are weak features.
2. For `axil_nodes` the curves are far apart: **52% of survivors have 0 positive nodes, against 23.5% of those who died**, and 84% of survivors have 5 or fewer nodes.
3. The highest node counts occur in both classes (46 nodes in a survivor, 52 in a patient who died), so **no single node count works as a hard cut-off**.

4.3 Box plot

A box plot shows the median (line), the quartiles (box) and the range (whiskers). The dots on top are the individual patients, so the box and the raw data can be read together.

History: the range-bar method was introduced by Mary Eleanor Spear in *Charting Statistics* (1952); the box-and-whisker plot by John Tukey in 1970, who later wrote about it in *Exploratory Data Analysis* (1977).

```
fig, axes = plt.subplots(1, 3, figsize=(16, 5))
for ax, feature in zip(axes, FEATURES):
    sns.boxplot(data=haberman, x=TARGET, y=feature, hue=TARGET, order=ORDER, palette=PALETTE, width=0.5,
                showfliers=False, boxprops=dict(alpha=0.55), linewidth=1.4, legend=False, ax=ax)
    sns.stripplot(data=haberman, x=TARGET, y=feature, order=ORDER, color=INK, size=3, alpha=0.35, jitter=0.22, ax=ax)
    ax.set(title=FEATURE_LABEL[feature].split(" (")[0], xlabel="", ylabel=FEATURE_LABEL[feature])
    ax.set_xticks(range(2), [NICE[o] for o in ORDER])
    ax.grid(axis="x", visible=False)
headline(fig, "Box plot of each feature by survival status")
plt.tight_layout()
plt.show()
```



Observations

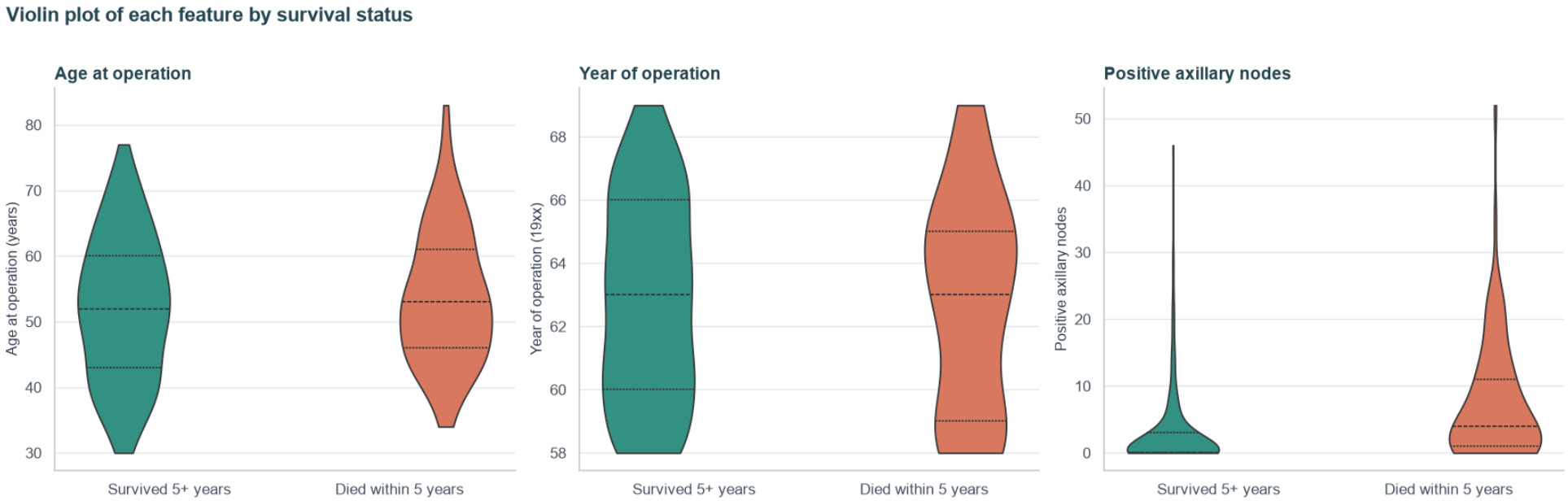
1. Survivors: `axil_nodes` has a median of 0 and a 75th percentile of 3.
2. Non-survivors: 25th percentile 1, median 4, 75th percentile 11.
3. The boxes for `age` and `operation_year` overlap almost completely.

4.4 Violin plot

A violin plot combines a box plot with a rotated density curve, so the *shape* of the distribution is visible too: the wider the violin, the more patients have that value. The dotted lines inside mark the quartiles.

History: proposed in 1997 by Jerry L. Hintze and Ray D. Nelson to show more information than the box plots that John Tukey created in 1977.

```
fig, axes = plt.subplots(1, 3, figsize=(16, 5))
for ax, feature in zip(axes, FEATURES):
    sns.violinplot(data=haberman, x=TARGET, y=feature, hue=TARGET, order=ORDER, palette=PALETTE, inner="quartile",
                   cut=0, linewidth=1.2, saturation=0.8, legend=False, ax=ax)
    ax.set(title=FEATURE_LABEL[feature].split(" (")[0], xlabel="", ylabel=FEATURE_LABEL[feature])
    ax.set_xticks(range(2), [NICE[o] for o in ORDER])
    ax.grid(axis="x", visible=False)
headline(fig, "Violin plot of each feature by survival status")
plt.tight_layout()
plt.show()
```



Observations

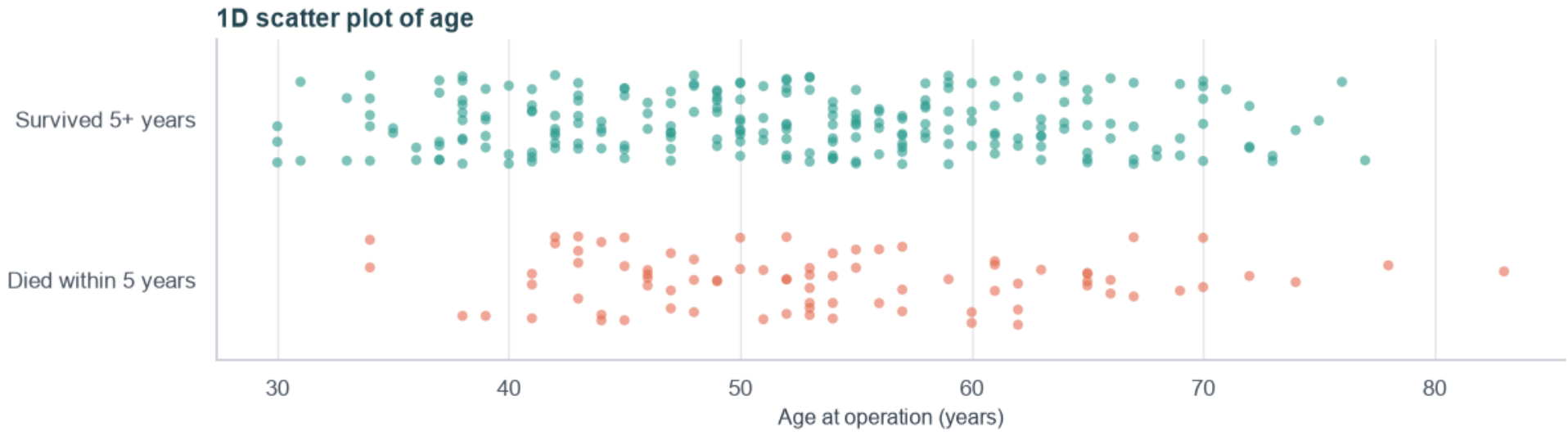
1. Survivors' `axil_nodes` is very dense between 0 and 5, with a thin tail.
2. Non-survivors' `axil_nodes` is spread much wider, so a high count points towards the "died" class.
3. The `age` violins differ only slightly: the non-survivor distribution sits a little higher (mean age 53.7 against 52.0), a small difference compared with the spread of ages.

4.5 1D scatter plot

Every patient is one dot on the age axis, one row per class. A little random vertical "jitter" stops the dots from sitting on top of each other.

```
fig, ax = plt.subplots(figsize=(11, 3.2))
sns.stripplot(data=haberman, x="age", y=TARGET, hue=TARGET, order=ORDER, hue_order=ORDER, palette=PALETTE,
              size=5, alpha=0.6, jitter=0.28, legend=False, ax=ax)
ax.set_yticks(range(2), [NICE[o] for o in ORDER])
ax.set(xlabel=FEATURE_LABEL["age"], ylabel="", title="1D scatter plot of age")
ax.grid(axis="y", visible=False)
```

```
plt.tight_layout()
plt.show()
```



Observation: both rows cover the same age range (about 30–80) and are densest between roughly 40 and 65, so age alone does not tell the classes apart.

4.6 Survival rate by number of positive nodes

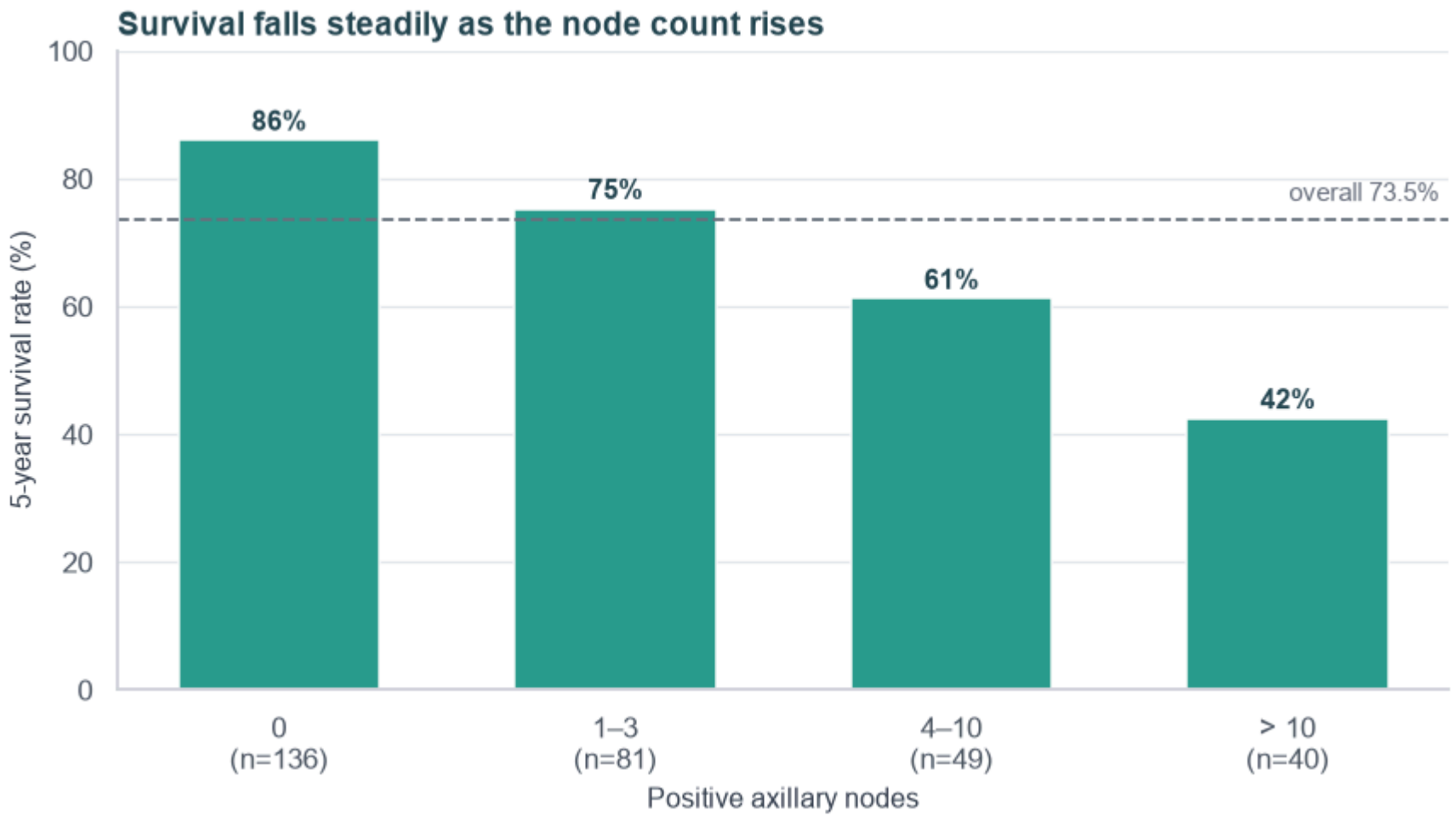
Grouping `axil_nodes` into bands makes the trend easy to see. The `n` under each bar is the number of patients in that band.

```
bands = pd.cut(haberman["axil_nodes"], bins=[-1, 0, 3, 10, 100], labels=["0", "1–3", "4–10", "> 10"])
rate = (haberman.assign(band=bands)
               .groupby("band", observed=True)[TARGET]
               .agg(patients="size", survival_rate=lambda s: (s == "yes").mean()))
display(rate.round(3))

fig, ax = plt.subplots(figsize=(8, 4.6))
bars = ax.bar([f"{band}\n(n={n})" for band, n in zip(rate.index, rate["patients"])],
              rate["survival_rate"] * 100, color=YES, width=0.6)
ax.axhline(73.5, color=MUTED, linestyle="--", linewidth=1.2)
ax.text(3.45, 76.5, "overall 73.5%", color=MUTED, ha="right", fontsize=9.5)
for bar, value in zip(bars, rate["survival_rate"] * 100):
    ax.text(bar.get_x() + bar.get_width() / 2, value + 1.5, f"{value:.0f}%", ha="center", color=INK, fontweight="bold")
ax.set(ylim=(0, 100), xlabel="Positive axillary nodes", ylabel="5-year survival rate (%)",
```

```
      title="Survival falls steadily as the node count rises")
ax.grid(axis="x", visible=False)
plt.tight_layout()
plt.show()
```

	patients	survival_rate
band		
0	136	0.860
1–3	81	0.753
4–10	49	0.612
> 10	40	0.425



Observation: the 5-year survival rate falls steadily as the number of positive nodes rises: about 86% with 0 nodes, 75% with 1–3, 61% with 4–10 and 42% with more than 10 (the overall rate is 73.5%).

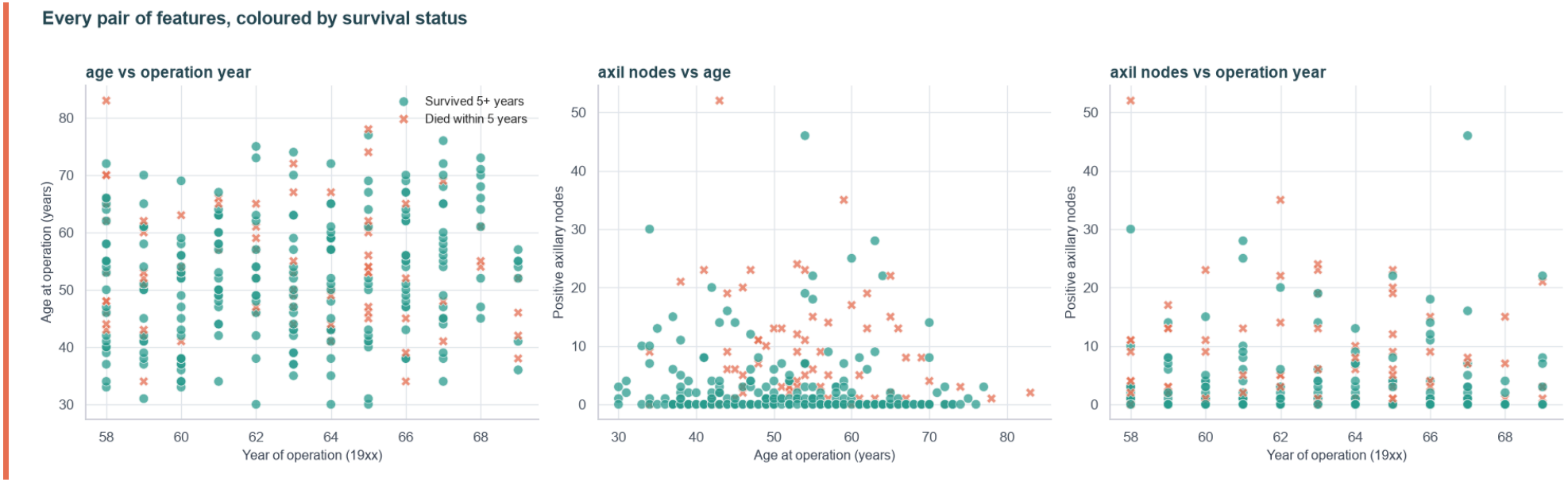
5 · Bivariate analysis

Bivariate analysis looks at **two variables together** to see whether a relationship exists between them. With 3 features there are 3C2 = **3 pairs**: (`operation_year` , `age`), (`age` , `axil_nodes`) and (`operation_year` , `axil_nodes`). The class label is not counted as a feature.

5.1 2D scatter plots

Colour **and** marker shape show the class, so the plots stay readable in black and white. If the two classes formed separate clouds, that pair of features would be useful for classification.

```
pairs = [("operation_year", "age"), ("age", "axil_nodes"), ("operation_year", "axil_nodes")]
fig, axes = plt.subplots(1, 3, figsize=(17, 5))
for k, (ax, (x, y)) in enumerate(zip(axes, pairs)):
    sns.scatterplot(data=haberman, x=x, y=y, hue=TARGET, style=TARGET, hue_order=ORDER, style_order=ORDER,
                    palette=PALETTE, markers={"yes": "o", "no": "X"}, s=55, alpha=0.75, edgecolor="white",
                    linewidth=0.5, legend=(k == 0), ax=ax)
    ax.set(title=f"{y.replace('_', ' ')} vs {x.replace('_', ' ')}", xlabel=FEATURE_LABEL[x], ylabel=FEATURE_LABEL[y])
nice_legend(axes[0].get_legend())
headline(fig, "Every pair of features, coloured by survival status")
plt.tight_layout()
plt.show()
```

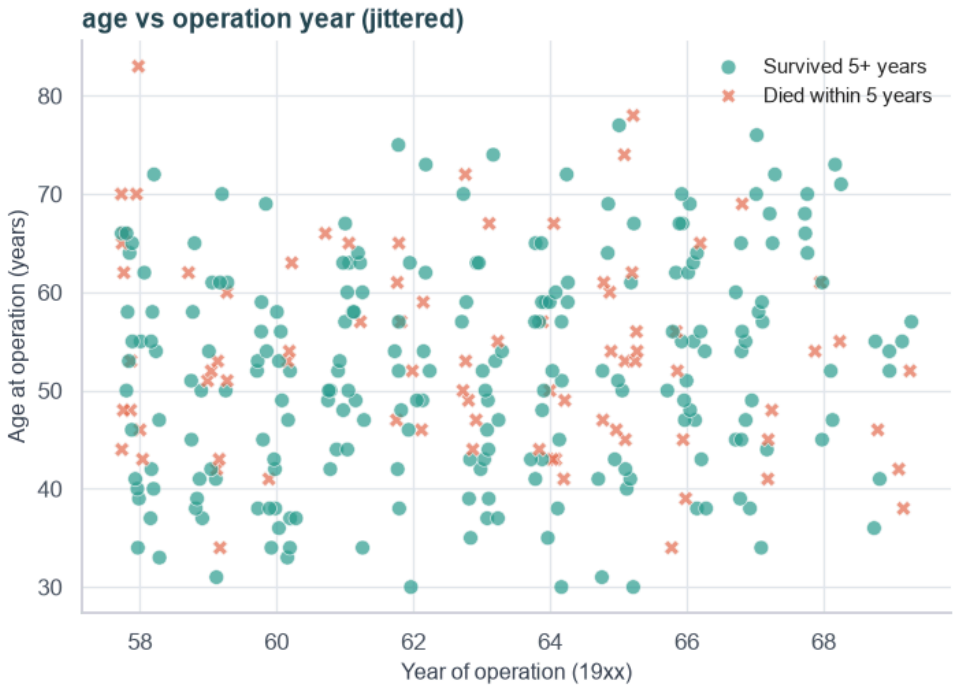


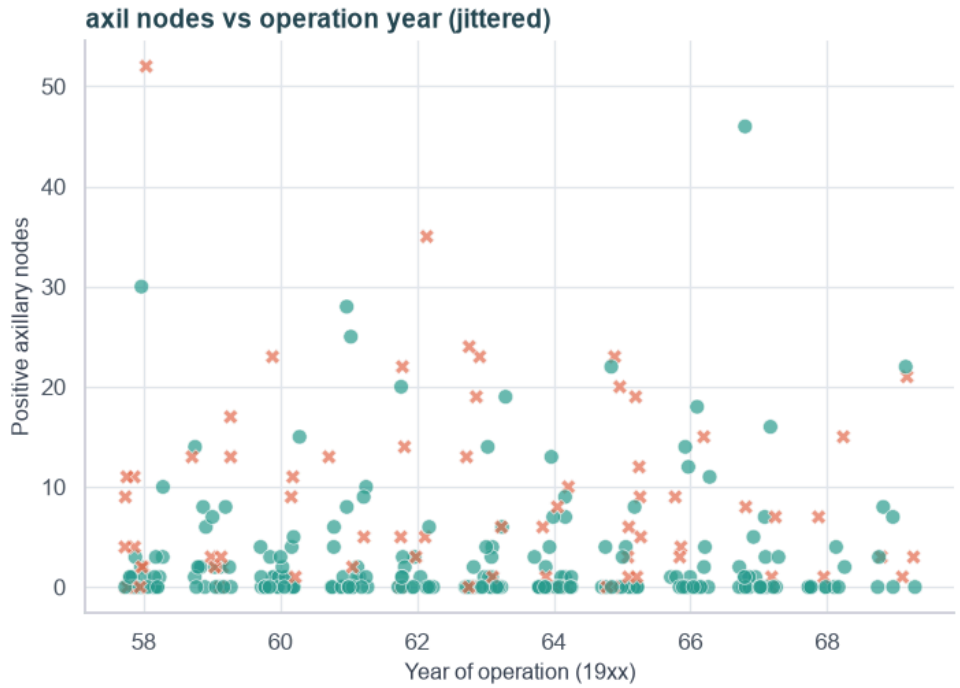
Observations

1. Most patients have 0 positive nodes, so the points pile up along the bottom of the plots that use `axil_nodes`.
2. `operation_year` is a whole number, so its points form vertical stripes and many overlap (see the jittered version below).
3. In every pair the two classes are mixed. **No two-feature combination separates the classes cleanly**, so a simple straight-line boundary would misclassify many patients.

```
# Same plots with a little horizontal jitter, so overlapping points become visible
rng = np.random.default_rng(42)
jittered = haberman.assign(operation_year_jitter=haberman["operation_year"] + rng.uniform(-0.3, 0.3, len(haberman)))

fig, axes = plt.subplots(1, 2, figsize=(13, 4.8))
for k, (ax, y) in enumerate(zip(axes, ["age", "axil_nodes"])):
    sns.scatterplot(data=jittered, x="operation_year_jitter", y=y, hue=TARGET, style=TARGET, hue_order=ORDER,
                    style_order=ORDER, palette=PALETTE, markers={"yes": "o", "no": "X"}, s=50, alpha=0.7,
                    edgecolor="white", linewidth=0.5, legend=(k == 0), ax=ax)
    ax.set(title=f"{y.replace('_', ' ')} vs operation year (jittered)", xlabel=FEATURE_LABEL["operation_year"], ylabel=FEATURE_LABEL[y])
nice_legend(axes[0].get_legend())
plt.tight_layout()
plt.show()
```





5.2 3D scatter plot

All three features at once. The plot is built with Plotly and saved as a **static image**, so it also appears in a PDF or HTML export of this notebook. An **interactive copy** (rotate, zoom, hide a class) is saved next to the notebook as `scatter_3d.html`; open it in a browser.

```
fig3d = px.scatter_3d(
    haberman,
    x="age",
    y="operation_year",
    z="axil_nodes",
    color=TARGET,
    opacity=0.8,
    color_discrete_map=PALETTE,
    category_orders={TARGET: ORDER},
    labels={
        "age": "age",
        "operation_year": "operation year",
        "axil_nodes": "axillary nodes",
        TARGET: "survival",
```

```
    },
    title="3D scatter plot of age, operation year and axillary nodes",
)
fig3d.update_traces(marker=dict(size=4, line=dict(width=0.4, color="white")))

LEGEND_NAMES = {"yes": "Survived 5+ years", "no": "Died within 5 years"}
fig3d.for_each_trace(lambda t: t.update(name=LEGEND_NAMES.get(t.name, t.name)))

fig3d.update_layout(
    width=1200,
    height=760,
    template="plotly_white",
    font=dict(size=12, color=INK),
    legend=dict(title="", yanchor="top", y=0.95, xanchor="left", x=0.02),
    scene_camera=dict(eye=dict(x=1.7, y=1.7, z=0.95)),
    margin=dict(l=0, r=0, t=50, b=0),
)

# 1) Interactive copy: open this file in a browser to rotate and zoom
fig3d.write_html("scatter_3d.html", include_plotlyjs="cdn")

# 2) Static copy: this is the one that appears in a PDF/HTML export
try:
    fig3d.write_image("scatter_3d.png", scale=2)  # needs the kaleido package
    display(Image("scatter_3d.png"))
except Exception as error:
    print("Could not save a static image:", error)
    print(
        "Run  %pip install -U kaleido  and restart the kernel. Showing the interactive plot instead."
    )
    fig3d.show()
```

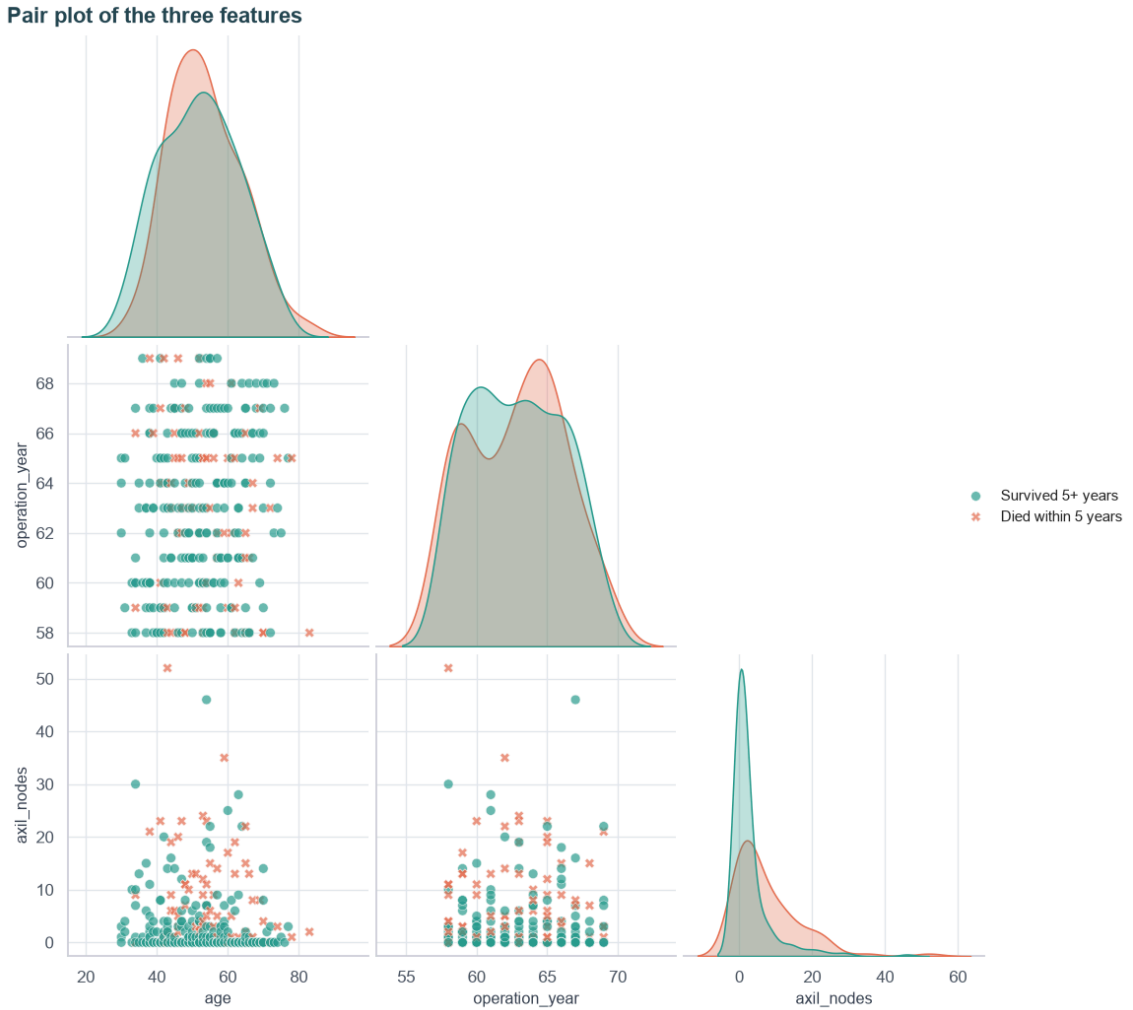


Observations: even with all three features, the classes are intermixed and no clean separating surface is visible. The only trend is along the `axil_nodes` axis, where the high points are mostly `no`.

5.3 Pair plot

A pair plot draws every pair of features in one grid. The diagonal shows each feature's own distribution per class, and the plots above and below the diagonal are mirror images (only the axes swap), so `corner=True` keeps just one triangle.

```
g = sns.pairplot(haberman, hue=TARGET, hue_order=ORDER, vars=FEATURES, palette=PALETTE, diag_kind="kde",
                 corner=True, height=3.1, markers=["o", "X"],
                 plot_kws=dict(alpha=0.7, s=40, edgecolor="white", linewidth=0.4),
                 diag_kws=dict(fill=True, alpha=0.3, common_norm=False))
nice_legend(getattr(g, "legend", None) or g.legend)
g.figure.suptitle("Pair plot of the three features", x=0.01, ha="left", fontsize=14.5, fontweight="bold", color=INK, y=1.01)
plt.show()
```

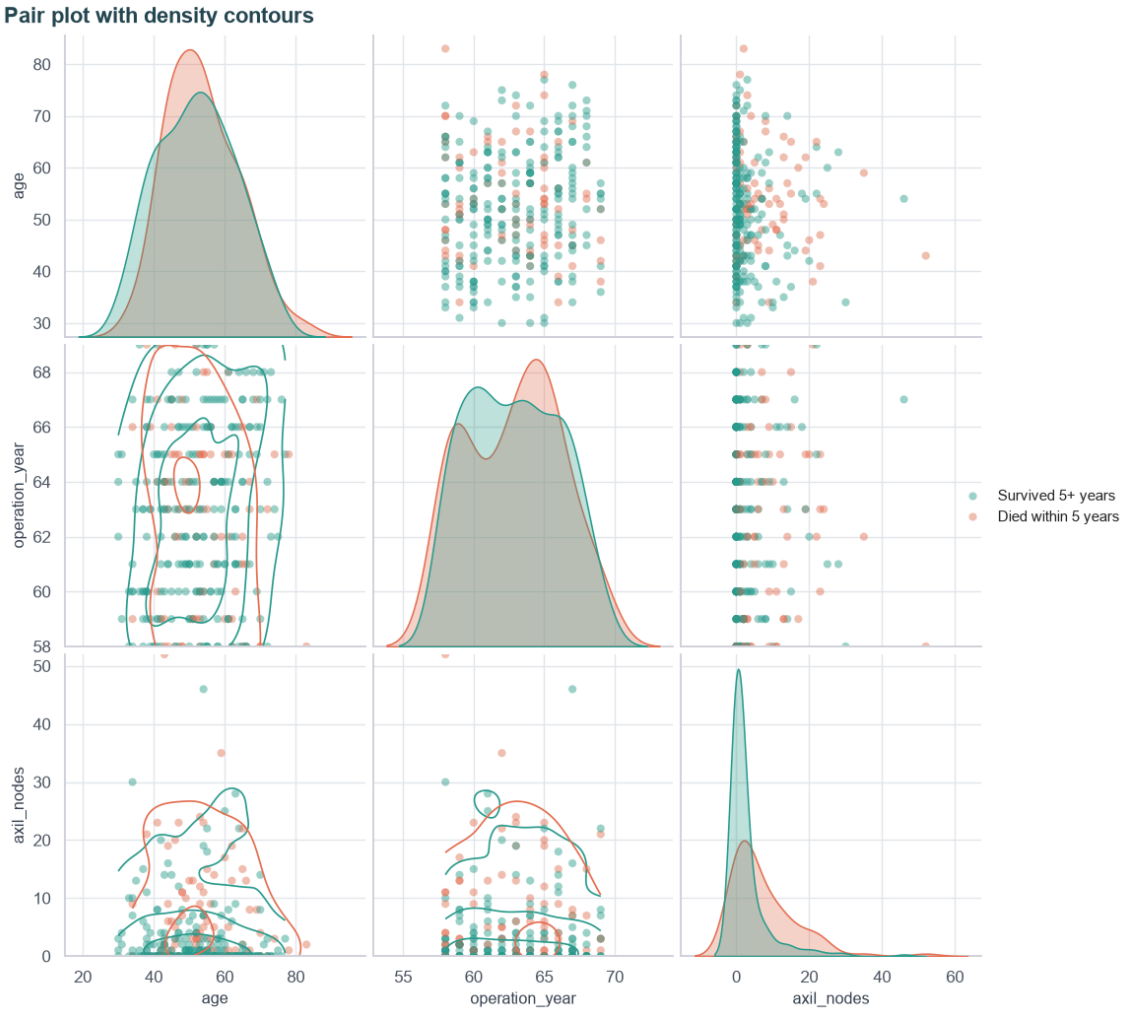


Observations

1. On the diagonal only `axil_nodes` shows two different shapes: survivors peak at 0, non-survivors are spread out.
2. In the lower triangle `axil_nodes` against `age` is the most informative pair: most survivors sit at 0 nodes at every age.
3. `age` against `operation_year` has overlapping classes everywhere, so it adds nothing.

The same grid with **density contours** added below the diagonal, which shows where each class is concentrated:

```
g = sns.pairplot(haberman, hue=TARGET, hue_order=ORDER, vars=FEATURES, palette=PALETTE, diag_kind="kde",
                 height=3.1, plot_kws=dict(alpha=0.45, s=28, edgecolor="none"),
                 diag_kws=dict(fill=True, alpha=0.3, common_norm=False))
g.map_lower(sns.kdeplot, levels=4, linewidths=1.1, cut=0)
nice_legend(getattr(g, "legend", None))
g.figure.suptitle("Pair plot with density contours", x=0.01, ha="left", fontsize=14.5, fontweight="bold", color=INK, y=1.01)
plt.show()
```

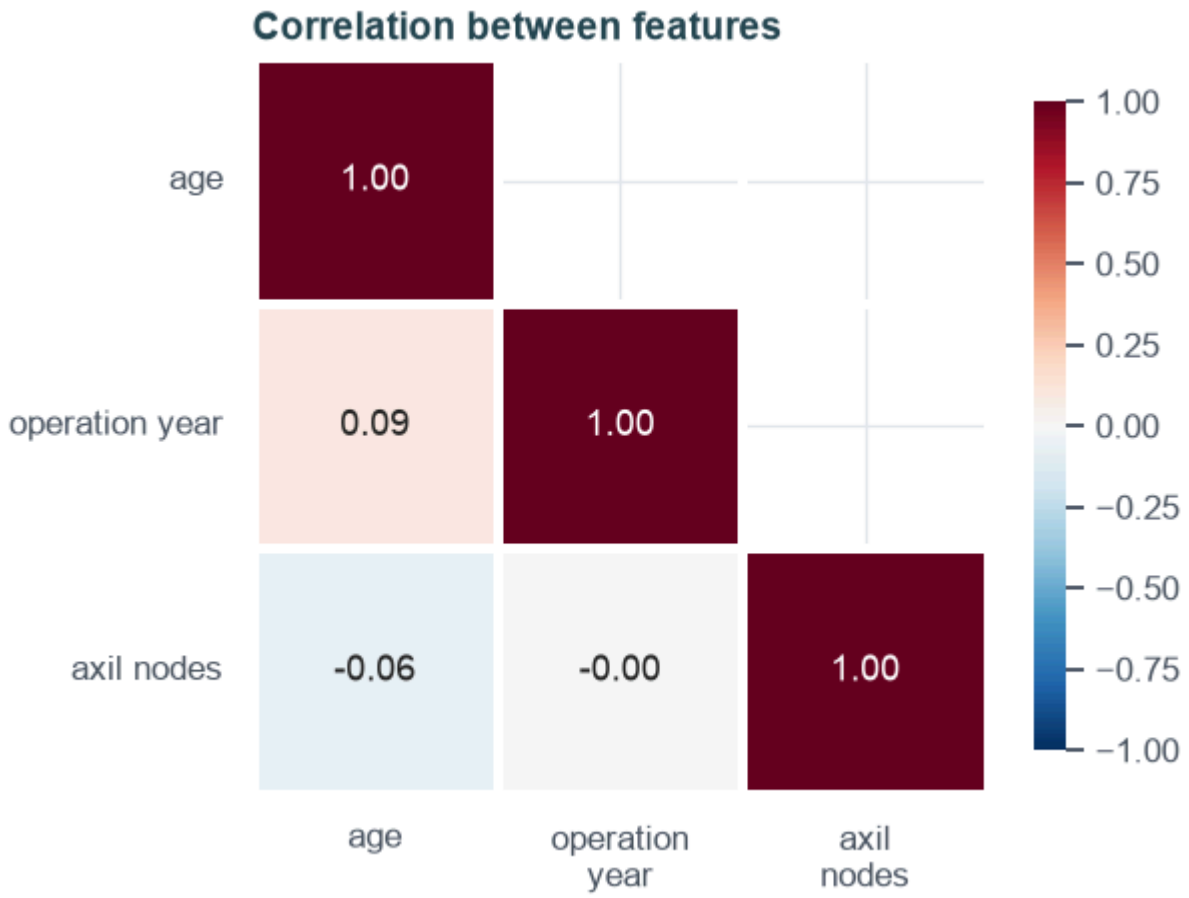


Observation: the contours of the two classes overlap in the `age` – `operation_year` panel, while in the panels with `axil_nodes` the `no` contours reach higher node counts than the `yes` contours.

5.4 Correlation between features

A correlation close to 0 means two features are not linearly related; close to ±1 means they move together.

```
corr = haberman[FEATURES].corr()
mask = np.triu(np.ones_like(corr, dtype=bool), k=1)
fig, ax = plt.subplots(figsize=(5.6, 4.6))
sns.heatmap(corr, mask=mask, annot=True, fmt=".2f", cmap="RdBu_r", vmin=-1, vmax=1, square=True,
            linewidths=2, linecolor="white", cbar_kws=dict(shrink=0.8), annot_kws=dict(size=12), ax=ax)
ax.set(title="Correlation between features")
ax.set_xticklabels([f.replace("_", "\n") for f in FEATURES], rotation=0)
ax.set_yticklabels([f.replace("_", " ") for f in FEATURES], rotation=0)
plt.tight_layout()
plt.show()
```



Observation: all correlations are close to 0 (age–operation_year 0.09, age–axil_nodes −0.06, operation_year–axil_nodes ≈ 0.00), so the features carry independent information and none is redundant.

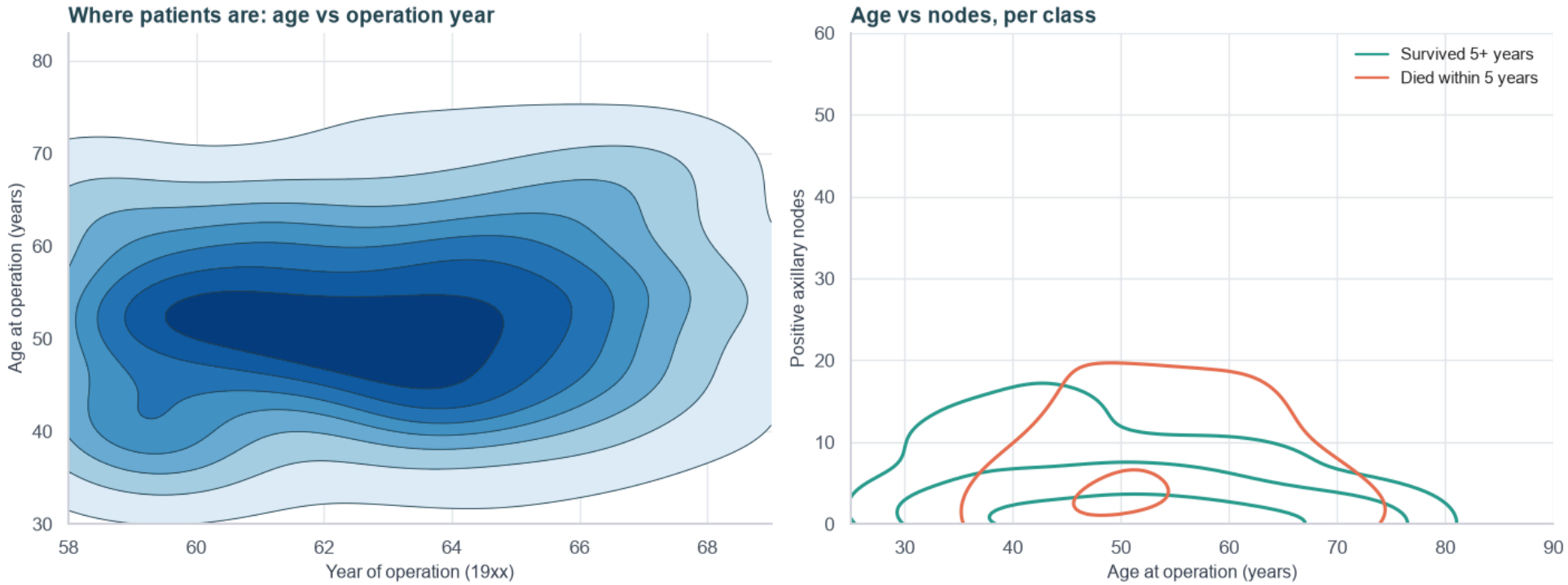
5.5 Density contours

Contour lines show where patients are concentrated. Left: all patients, age against operation year. Right: age against node count, one set of contours per class.

```
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
```

```
sns.kdeplot(data=haberman, x="operation_year", y="age", fill=True, cmap="Blues", thresh=0.05, levels=8, cut=0, ax=axes[0])
sns.kdeplot(data=haberman, x="operation_year", y="age", color=INK, linewidths=0.6, levels=8, cut=0, ax=axes[0])
axes[0].set(title="Where patients are: age vs operation year", xlabel=FEATURE_LABEL["operation_year"], ylabel=FEATURE_LABEL["age"])

sns.kdeplot(data=haberman, x="age", y="axil_nodes", hue=TARGET, hue_order=ORDER, palette=PALETTE, levels=4,
            thresh=0.15, linewidths=2, clip=((25, 90), (0, 60)), ax=axes[1])
axes[1].set(title="Age vs nodes, per class", xlabel=FEATURE_LABEL["age"], ylabel=FEATURE_LABEL["axil_nodes"])
nice_legend(axes[1].get_legend())
plt.tight_layout()
plt.show()
```



Observations: the middle 50% of patients were aged about 44–61 and were operated on between 1960 and 1966; operations became less frequent in 1968–1969 (13 and 11 patients, against 23–36 in each earlier year). In the right panel the contours of the `no` class extend to higher node counts than those of the `yes` class.

---

6 · Key findings, observations and conclusions

6.1 The picture in one figure

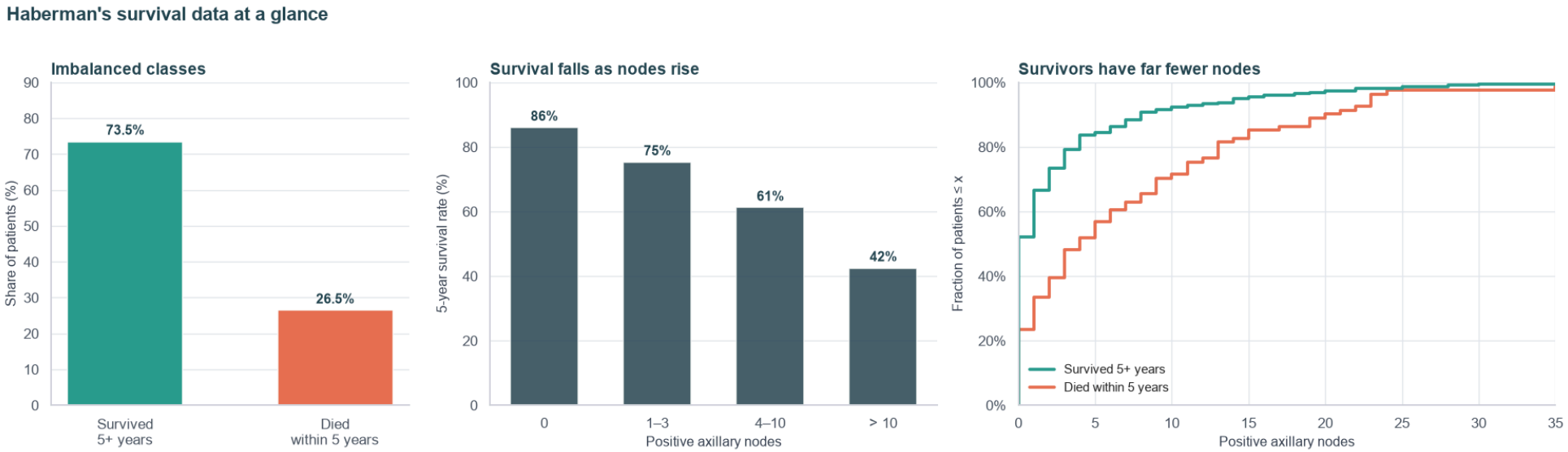
```
fig, axes = plt.subplots(1, 3, figsize=(17, 4.8), gridspec_kw={"width_ratios": [0.8, 1, 1.2]})

# (a) class balance
share = haberman[TARGET].value_counts(normalize=True).reindex(ORDER) * 100
bars = axes[0].bar([NICE[o].replace(" ", "\n", 1) for o in ORDER], share.values, color=[YES, NO], width=0.55)
axes[0].bar_label(bars, fmt="%.1f%%", padding=3, color=INK, fontweight="bold")
axes[0].set(title="Imbalanced classes", ylim=(0, 90), ylabel="Share of patients (%)")
axes[0].grid(axis="x", visible=False)

# (b) survival rate by node band
bars = axes[1].bar(rate.index.astype(str), rate["survival_rate"] * 100, color=INK, width=0.6, alpha=0.85)
axes[1].bar_label(bars, fmt="%.0f%%", padding=3, color=INK, fontweight="bold")
axes[1].set(title="Survival falls as nodes rise", ylim=(0, 100), xlabel="Positive axillary nodes", ylabel="5-year survival rate (%)")
axes[1].grid(axis="x", visible=False)

# (c) CDF of nodes
sns.ecdfplot(data=haberman, x="axil_nodes", hue=TARGET, hue_order=ORDER, palette=PALETTE, linewidth=2.4, ax=axes[2])
axes[2].set(title="Survivors have far fewer nodes", xlabel=FEATURE_LABEL["axil_nodes"], ylabel="Fraction of patients ≤ x", xlim=(0, 35))
axes[2].yaxis.set_major_formatter(PercentFormatter(1.0))
nice_legend(axes[2].get_legend())

headline(fig, "Haberman's survival data at a glance")
plt.tight_layout()
plt.savefig("key_findings.png", bbox_inches="tight")
plt.show()
```



6.2 All observations

Number of points

- The dataset is **imbalanced** (about 73.5% `yes`, 26.5% `no`) but complete, with no missing values.
- `survival_status` was an integer, so it was converted to the categories `yes` (survived 5 years or longer) and `no` (died within 5 years).

PDF / CDF

- The distributions of `age` and `operation_year` are similar for both classes. `axil_nodes` is the only feature whose distribution clearly differs.
- The survival rate is high for patients with few positive nodes.

Box plot and violin plot

- Survivors: median `axil_nodes` is 0 and the 75th percentile is 3.
- Non-survivors: the 25th percentile is 1, the median is 4 and the 75th percentile is 11.

- The `age` and `operation_year` boxes overlap almost completely.

Scatter plots, pair plot and contours

- Most patients have 0 positive nodes, and the classes overlap in every pair of features.
- Only `axil_nodes` shows different shapes for the two classes.

6.3 Conclusions

1. The data is **imbalanced**: about 73.5% of patients survived 5 years or longer and 26.5% died within 5 years. Plain accuracy would be misleading for any model built on it.
2. `axil_nodes` **is the only useful single feature.** Survivors have a mean of about 2.8 positive nodes and non-survivors about 7.5. The median is 0 for survivors and 4 for non-survivors, and about 52% of survivors have 0 nodes against about 23.5% of non-survivors.
3. The survival rate falls as the number of nodes rises: about 86% with 0 nodes, 75% with 1–3, 61% with 4–10 and 42% with more than 10.
4. `age` **and** `operation_year` **are poor features.** Their distributions are almost the same for both classes (mean age about 52.0 for survivors and 53.7 for non-survivors).
5. In the scatter plots and the pair plots the two classes **overlap heavily**, so no pair of features separates them cleanly.
6. A high node count is not a rule: one patient with 46 positive nodes survived and one with 52 did not, so there is no single cut-off value.
7. The features are almost uncorrelated with each other (the largest correlation is about 0.09), so they carry independent information.
8. Overall, these three features give only **moderate predictive power**.

6.4 Possible next steps

- Train a simple classifier (logistic regression, decision tree) and compare it with the 73.5% "always predict survived" baseline.
- Judge models with precision, recall and ROC-AUC, because accuracy hides the imbalance.
- Use stratified cross-validation, since the dataset has only 306 patients.